

방 사진 기반 2D 평면도 자동 생성

목차 (Table of Contents)

1. 전체 EPIC
2. 스프린트 계획
 - 스프린트 1주차 (2025.07.08 ~ 07.12)
 - 주간 목표
 - 일별 스크럼
 - 7월 8일 (화)
 - 7월 9일 (수)
 - 7월 10일 (목)
 - 7월 11일 (금)
 - 스프린트 2주차 (2025.07.14 ~ 07.18)
 - 주간 목표
 - 일별 스크럼
 - 7월 14일 (월)
 - 7월 15일 (화)

1. 전체 EPIC

사용자가 업로드한 방 사진을 기반으로 YOLOv11, MiDaS, RoomNet 등의 AI 모델을 활용해 가구를 감지하고, 깊이-레이아웃 정보를 추정하여 실제 방의 평면도(2D) 구조를 자동으로 시각화하는 웹 서비스를 개발한다. 주요 작업에는 모델 연동, 프론트 UI/UX 구성, API 구축, 도커/쿠버네티스 배포 및 CI/CD 자동화가 포함된다.

EPIC-1: AI 기반 분석 파이프라인 구성

EPIC-2: MiDaS 기반 실측 거리 추정 기능 구현

EPIC-3: 정확도 향상 및 안정화 작업

EPIC-4: 2D 평면도 시각화 및 UI 통합

2. 스프린트 계획

2.1. 스프린트 1주차 (2025.07.08 ~ 07.12)

2.1.1. 1주차 EPIC : AI 기반 분석 파이프라인 구성

YOLOv11, MiDaS, RoomNet 등 AI 모델을 테스트하고, 업로드된 방 사진을 분석하는 초기 파이프라인을 구성한다.

FastAPI 서버 구조 설계, 사진 업로드 UI, 층고 입력 필드 구축까지 포함.

2.1.2. 일별 스크럼

2.1.2.1. 7월 8일 (화)

- **어제 한 일:** (N/A – 주간 시작일)
- **오늘 할 일:**
 - AI: YOLOv11 사전 학습된 모델 로딩 및 샘플 이미지 추론 테스트
 - FastAPI: `/api/layout/detect` 기본 스켈레톤 구성
 - 프론트엔드: 사진 업로드 컴포넌트 및 층고 입력 필드 UI 구현 시작
 -
- **이슈:** -
- **블로커:** -

2.1.2.2. 7월 9일 (수)

- **어제 한 일:**
 - YOLOv11 모델 추론 테스트 성공
 - FastAPI 기본 구조 및 업로드 엔드포인트 생성
 - 프론트엔드 파일 업로드 드래그앤드롭 UI 구축 완료
- **오늘 할 일:**
 - FastAPI `/analyze-room` 에 입력 포인트 (ceiling_y, floor_y, left_x, right_x) 기반 room size 계산 적용
 - OpenCV 활용한 벽 높이 픽셀 계산 로직 추가
 - 좌우 벽 포인트 기반 가로 너비 계산 로직 적용
 - 프론트엔드에서 방 크기(cm 단위) 및 감지 결과 시각화 연동 검증
- **이슈:**
 - 층고 기반 환산 시 기준 객체(Bed 등)가 없을 경우 좌우 벽 포인트로 보완하는 방식으로 결정 중
- **블로커:**
 - 이미지가 왜곡되어 찍혔을 경우 벽 높이 픽셀 길이 기준 환산의 정밀도 저하 우려 있음 → 추후 보정 알고리즘 필요

2.1.2.3. 7월 10일 (목)

- **어제 한 일:**

- 층고값 기반 비율 환산 로직 기본 구현
- 프론트엔드 층고 입력 필드 + 유효성 검사 구현

• 오늘 할 일:

- FastAPI에서 `/estimate-room-size`` API 설계 시작
 - 사용자로부터 4개 지점(바닥, 천장, 좌/우 벽)의 Pixel + Depth 입력 받아 크기 추정 구조 정의
- MiDaS 기반 depth 추정값을 3D 좌표로 환산하는 함수 설계 (`to_3d_point`)

• 이슈:

- 프론트에서 수집된 좌표가 depth와 함께 정확히 전달되지 않음
- 클릭한 좌표에 따라 추정되는 크기가 편차 발생

• 블로커:

- MiDaS가 제공하는 깊이값이 정규화된 상대값이기 때문에 실제 단위 거리 환산 불가능
- 기준이 되는 실제 층고가 주어지지 않으면 스케일 보정이 어려움

2.1.2.4. 7월 11일 (금)

• 어제 한 일:

- ```/estimate-room-size`` API 1차 버전 완성
 - 입력된 4점(Point3D) → MiDaS 기반 3D 좌표로 변환
 - 3D 거리 계산 후, 실제 층고 230cm를 기준으로 스케일 보정
 - 가로·세로 크기(cm), 기존 2D 방식 대비 개선률 포함 응답 구성

• 오늘 할 일:

- ```/get-depth-at-point`` , `/depth-distance`` 등 보조 API 연결 및 테스트
- `FastAPI 로그 개선 (좌표 정보, 변환 과정 로그 출력)
- `MiDaS depth 변환 공식을 튜닝하여 상대 오차 보정 시도

• 이슈:

- 동일한 이미지라도 클릭 위치에 따라 깊이값 편차 심함
- 정규화된 좌표 → 실좌표 변환시 focal length 가정이 맞지 않으면 왜곡 발생

• 블로커:

2.2. 스프린트 2주차 (2025.07.14 ~ 07.18)

2.2.1. 2주차 EPIC : MiDaS 기반 실측 거리 추정 기능 구현

MiDaS depth 결과를 이용해 사용자가 선택한 좌표들로부터 실제 방 크기를 추정하고, 기존 방식 대비 개선된 정확도를 확보한다.

스케일 보정, 3D 좌표 계산, 거리 비교 등을 포함한다.

2.2.2. 일별 스크립

2.2.2.1. 7월 14일 (월)

- 어제 한 일:

- ``/estimate-room-size` API 고도화
 - depth 기반 3D 변환 + 3D 거리 추정 + 스케일 보정 흐름 정리
 - 기존 2D 방식과의 비교값 제공(`improvement_percent`)
 - depth range, 시점 왜곡 강도(`perspective_strength`) 판단 로직 추가

- 오늘 할 일:

- 프론트엔드와 `/estimate-room-size` 연결
 - 클릭한 좌표와 depth를 백엔드에 전달하여 실시간 방 크기 계산
- `/depth-distance` 와 로직 공통화 및 모듈 분리 작업
- Canvas에 2D 평면도 형태로 시각화 시작

- 이슈:

- 입력된 4점의 정렬이나 각도가 실제 방 구조와 다를 경우 결과가 왜곡됨
- depth range가 좁은 경우 실측 정확도가 떨어짐

- 블로커:

2.2.2.2. 7월 15일 (화)

- 어제 한 일:

- `/estimate-room-size` 에 3D 기반 좌표 계산 및 실측 보정 로직 적용 완료
 - `to_3d_point()` 로 MiDaS depth → 3D world coordinate 변환 구현
 - 기준층고(2.3m)를 기준으로 scale factor 계산 → width/depth 보정
- 2D vs 3D 계산 결과 비교 로그 출력 (개선율 계산 포함)

- 방 크기 추정 결과에 `perspective_strength`, `depth_range`, 스케일 팩터 등 정보 추가

- **오늘 할 일:**

- 정확도 향상을 위한 구조 개선
 - 다양한 계수 테스트 및 상대 오차 분석
- MiDaS가 출력하는 depth의 상대성 극복을 위한 벤치마크 테스트
 - 다양한 방 이미지에서 실제값 대비 예측값 비교
 - 평균 scale factor, 편차 분석 → 보정 근거 수립

- **이슈:**

- **블로커:**

2.2.2.3. 7월 16일 (수)

- **어제 한 일:**

- 정확도 향상을 위한 구조 개선
- MiDaS가 출력하는 depth의 상대성 극복을 위한 벤치마크 테스트

- **오늘 할 일:**

- 클릭 기반 거리 실험 및 로그 수집
 - `/get-depth-at-point` API 사용하여 다수의 클릭 지점 depth 수집
 - 실제 거리와 비교해 적절한 보정 수식 실험 ($a/(depth+b)$ 등)
 - 회귀 방식 2~3가지 실험 및 로그 저장
- 3D 거리 vs 2D 거리 비교
 - `/depth-distance` API로 보정 전후 거리 차이 확인
 - 같은 지점 쌍에 대해 2D 거리와 3D 거리 차이 시각화

- **이슈:**

- **블로커:**

2.2.2.4. 7월 17일 (목)

- **어제 한 일:**

- 클릭 기반 거리 실험 및 로그 수집 완료
- 2D/3D 거리 비교 데이터 확보

- **오늘 할 일:**

- 수집된 로그 기반으로 최적의 보정 수식 도출
- `/estimate-room-size` API에 최종 보정 로직 적용

- 프론트엔드에서 보정 전/후 거리 비교 시각화 기능 구현
- 이슈:
 - 특정 구도(예: 광각)에서 왜곡 보정 효과 미미
- 블로커:

2.2.2.5. 7월 18일 (금)

- 어제 한 일:
 - API에 최종 보정 로직 적용 완료
 - 프론트엔드 비교 시각화 기능 1차 구현
- 오늘 할 일:
 - 다양한 테스트 이미지로 최종 정확도 검증
 - 2주차 스프린트 결과 정리 및 회고
 - 3주차 계획 구체화 (캔버스 시각화)
- 이슈:
- 블로커:

2.2.2.6. 7월 21일 (월)

- 어제 한 일: 2주차 스프린트 결과 정리 및 회고.
- 오늘 할 일:
 - 현재 구현된 방 측정 및 시각화 기능(이미지 업로드, 깊이 맵 생성, 방 크기 추정, 2D/3D 시각화)에 대한 전반적인 검토.
 - 측정된 방 데이터를 활용한 2D 평면도 인터랙션 강화 및 3D 뷰어 시각적 개선 계획 수립.
 - 백엔드에서 평면도 데이터(방 외곽선, 가구 위치 등)를 JSON으로 반환하는 API 스텁레톤 구성 (인테리어 디자인 활용 목적).
 - 프론트엔드에서 2D 캔버스 라이브러리(Konva.js 또는 Fabric.js) 설치 및 기본 설정.
- 이슈:
- 블로커: -

1주차 스크럼 (7월 8일 ~ 7월 12일) - 완료

 **주요 목표:** AI 모델(MiDaS) 연동 및 핵심 API/UI 개발

 **핵심 작업:**

- AI/백엔드: MiDaS(small) 모델을 이용한 /depth-map API 구현 완료.
- 프론트엔드: Vite, React, Tailwind CSS 기반 프로젝트 설정 및 ImageUploader 컴포넌트 구현 완료.
- 논의 사항: 초기 계획의 RoomNet, YOLOv11은 리소스 및 복잡성을 고려하여 MiDaS 우선 적용으로 방향 수정.

⚠ 이슈/블로커: 없음.

📅 2주차 스크럼 (7월 14일 ~ 7월 18일) - 완료

🎯 주요 목표: MiDaS 기반 원근 보정 및 크기 추정 로직 고도화

🔧 핵심 작업:

- **백엔드:** 4개 좌표와 층고를 이용해 원근을 보정하는 `/estimate-room-size` API 개발 완료.
- **백엔드:** 측정 결과의 신뢰도(confidence)를 계산하는 로직 추가.
- **프론트엔드:** `ImageClickArea` 컴포넌트에서 4개 점을 클릭하여 서버로 전송하고, 측정 결과를 받아 오는 전체 플로우 완성.
- **논의 사항:** MiDaS의 상대 깊이 한계를 극복하기 위해, '층고 기반 스케일링 + 상대 깊이 이용한 원근 보정'이라는 독자적 접근 방식 채택.

⚠ 이슈/블로커: MiDaS의 상대적 깊이 값을 절대 거리로 변환하는 것이 어려웠으나, 원근 보정 방식으로 문제 해결.

📅 3주차 스크럼 (7월 21일 ~ 7월 27일) - 계획

🎯 주요 목표: 2D/3D 결과 시각화 및 UI 완성

🔧 핵심 작업:

- 측정 결과를 2D 평면도(`RoomResult`)와 3D 뷰어(`RoomBox`)로 시각화.
- `@react-three/fiber`를 사용하여 3D 뷰에 `OrbitControls`(궤도 제어) 적용.
- 2D/3D 뷰를 전환하는 탭 UI 구현 및 전체적인 사용자 경험(UX) 개선.

💬 논의 사항:

- AI 측정의 한계를 보완하기 위한 사용자 편집 기능(결과 미세 조정) 도입 여부.
- 3D 렌더링 성능 최적화 방안.

⚠ 이슈/블로커:

- (예상) 다양한 디바이스에서의 3D 뷰 성능 저하 가능성.

📅 4주차 스크럼 (7월 28일 ~ 8월 3일) - 계획

🎯 주요 목표: 로컬 실행 환경 완성 (Docker 기반) 및 최적화

🔧 핵심 작업:

- FastAPI, OpenCV, PyTorch(CPU) 기반 백엔드 Dockerfile 작성.
- React, Vite 기반 프론트엔드 Dockerfile 작성.
- docker-compose 를 사용하여 로컬에서 전체 애플리케이션 실행 환경 구성.

💬 논의 사항:

- 모델 캐시 적용 범위 및 메모리 최적화.
- 이미지 업로드 용량 제한 및 관련 에러 처리.

⚠️ 이슈/블로커:

- AI 모델을 포함한 Docker 이미지의 용량 문제.

📅 5주차 스크럼 (8월 4일 ~ 8월 10일) - 계획

🎯 주요 목표: Kubernetes 환경 배포 및 통합 테스트

🔧 핵심 작업:

- K8s Deployment, Service, Ingress 리소스 YAML 파일 작성.
- 실제 도메인 연결 및 Ingress를 통한 외부 트래픽 노출.
- 전체 기능(업로드-분석-시각화) 통합 테스트.

💬 논의 사항:

- 모델 파일을 이미지에 포함할지, 외부 스토리지(S3 등)에서 로드할지 결정.
- Pod 리소스(CPU, Memory) 요청 및 제한 설정.

⚠️ 이슈/블로커:

- (예상) 클러스터 환경에서의 파일 시스템 접근 및 권한 문제.

📅 6주차 스크럼 (8월 11일 ~ 8월 17일) - 계획

🎯 주요 목표: CI/CD 자동 배포 파이프라인 구축

🔧 핵심 작업:

- GitHub Actions를 사용하여 main 브랜치 push 시 자동으로 Docker 이미지 빌드 및 K8s에 배포하는 워크플로우 구성.
- 상태 모니터링을 위한 Prometheus/Grafana 도입 검토.
- Ingress에 TLS를 적용하여 HTTPS 통신 설정.

💬 논의 사항:

- GitHub Secrets를 이용한 민감 정보(API 키, 인증서 등) 관리.
- 배포 실패 시 Rollback 전략 수립.

⚠ 이슈/블로커:

- (예상) CI/CD 파이프라인 내에서 K8s 클러스터 인증 및 통신 설정.

📅 7주차 스크럼 (8월 18일 ~ 8월 20일) - 계획

🎯 주요 목표: 최종 시연 준비 및 문서화

🔧 핵심 작업:

- 전체 기능 시연 시나리오 작성 및 리허설.
- 프로젝트 README.md 에 실행 방법 및 기술 아키텍처 상세히 기술.
- API 명세서 및 최종 발표 자료 작성.

💬 논의 사항:

- 발표 핵심: "MiDaS의 상대 깊이 한계를 어떻게 창의적으로 해결했는가?"를 기술적 차별점으로 강조.
- 시연 포인트: 2D와 3D 뷰를 함께 제공하여 사용자의 이해를 돕는 점을 부각.
- 최종 결과물 공유 또는 다운로드 기능 추가 여부.

⚠ 이슈/블로커:

- 발표 직전 발생할 수 있는 버그에 대한 비상 대응 계획 필요.

📖 스프린트 스크럼 요약 (1~7주차)

📅 1주차 (7/8~7/13)

날짜	할 일	담당	상태
7/8 (월)	YOLOv11, MiDaS, RoomNet 모델 PoC 테스트	AI	완료
7/9 (화)	/api/layout/detect API 스켈레톤 구성	백엔드	완료
7/10 (수)	업로드 UI 기획 및 캔버스 프레임워크 검토	프론트	완료
7/11 (목)	충고 기본값 처리 및 모델 연동 기획	백엔드	완료

날짜	할 일	담당	상태
7/12 (금)	기능 구성안 기반 와이어프레임 작성	프론트	완료



2주차 (7/14~7/20)

날짜	할 일	담당	상태
7/14 (월)	/estimate-room-size 3D 보정 로직 설계 및 구현 시작	백엔드	완료
7/15 (화)	좌표 기반 거리 계산 API 고도화 및 비교 로그 추가	백엔드	진행 중
7/16 (수)	층고 입력 필드 유효성 검사 및 클릭 이벤트 처리	프론트	예정
7/17 (목)	MiDaS 좌표 정규화 및 3D 변환 함수 보정	AI/백엔드	예정
7/18 (금)	업로드 이미지 해상도 자동 리사이징 처리	프론트/백엔드	예정
7/19 (토)	전체 API-UI 연동 점검 및 캔버스 렌더링 확인	전체	예정



3주차 (7/21~7/25)

날짜	할 일	담당	상태
7/21 (월)	평면도 캔버스 구성 및 좌표 → cm 환산 공식 적용	프론트	예정
7/22 (화)	RoomNet 실패 시 YOLO+OpenCV fallback 처리	백엔드	예정
7/23 (수)	전체 시각화 플로우 점검 및 에러 메시지 처리	프론트/백	예정
7/24 (목)	테스트 이미지 기반 결과 품질 검토	전체	예정
7/25 (금)	모델 추론시간 측정 및 성능 프로파일링	전체	예정



4주차 (7/28~8/1)

날짜	할 일	담당	상태
7/28 (월)	OpenCV/torch 포함 Dockerfile 작성	백엔드	예정
7/29 (화)	docker-compose로 로컬 실행환경 구성	전체	예정
7/30 (수)	모델 로딩 방식(Lazy vs Static) 비교 실험	AI	예정

날짜	할 일	담당	상태
7/31 (목)	캐시 전략 및 리소스 최적화 기획	전체	예정
8/1 (금)	로컬 통합 테스트 완료	전체	예정



5주차 (8/4~8/8)

날짜	할 일	담당	상태
8/4 (월)	K8s Deployment/Service YAML 작성	DevOps	예정
8/5 (화)	Ingress 설정 및 도메인 연동	DevOps	예정
8/6 (수)	앱 배포 및 외부 접속 확인	DevOps	예정
8/7 (목)	프론트-백엔드 연동 기능 점검	백엔드	예정
8/8 (금)	테스트 자동화 스크립트 적용	전체	예정



6주차 (8/11~8/15)

날짜	할 일	담당	상태
8/11 (월)	GitHub Actions 배포 자동화 구성 시작	DevOps	예정
8/12 (화)	Prometheus 등 상태 모니터링 도구 연동	DevOps	예정
8/13 (수)	Ingress 보안 설정(TLS 등) 적용	DevOps	예정
8/14 (목)	실시간 모니터링 및 성능 분석	전체	예정
8/15 (금)	Rollback 시나리오 테스트	전체	예정



7주차 (8/18~8/20)

날짜	할 일	담당	상태
8/18 (월)	통합 시연 리허설, 테스트 체크리스트 작성	전체	예정
8/19 (화)	발표용 슬라이드, 소개자료, API 명세서 작성	전체	예정
8/20 (수)	최종 발표 및 전체 기능 배포 완료	전체	예정

